SLLAB

A platform for running a college club sports team

Lucius Scala
University of California, San Diego · lscala@ucsd.edu
November 2026

Abstract. College club sports teams are run by students, with no staff, no budget for professional tools, and officers who change every year or two. SLLAB is a platform that gives a club the logistics of a staffed program: a schedule and availability app, SMS reminders, attendance tracking, a public website with live standings, and an automated game-film system built on a custom tracking camera mount. It is in daily use by the UC San Diego Men's Club Soccer team, and its iOS app is on the App Store. This write-up describes the system as built: its architecture, data model, the computer-vision and embedded control loop behind the camera, and the plan for turning one team's tools into a platform any club can use.

1 Motivation

A college club team does everything a varsity program does, such as scheduling practices, booking fields, travelling to tournaments, recruiting at tryouts and keeping a roster eligible, but students do all of it, in their spare time. The tools they use are general purpose: group chats for attendance, spreadsheets for rosters, a form for tryouts, a website nobody updates. Each works on its own, and nothing connects them. When officers graduate, much of what they knew leaves with them.

Two costs make this worse. Game film, which teams use to improve, usually means paying for a camera operator, or for a commercial auto-tracking camera at around $2,000. And because the tools are disconnected, the same information is entered in several places and drifts out of date.

The goal of SLLAB is one connected platform with one backend: the schedule a captain creates is the one players respond to, the one the public website shows, and the one game film is attached to. It should be simple enough that a club adopts it without a developer, and polished enough that the team feels like a program.

What exists

- **Fixture by SLLAB**, an iOS app for schedules, availability, reminders and attendance. Released on the App Store (version 1.2).
- **UCSDMCS by SLLAB**, the club's public website, with live standings, a calendar feed, a web version of Fixture, and an admin dashboard.
- **Film by SLLAB**, an iOS app that records games on a phone mounted on a motorized base, steered by on-device object detection. Working prototype.
- **Hardware by SLLAB**, the mount itself: ESP32 firmware, a custom PCB and 3D-printed parts, at about $400 in total.

2 System overview

Every part of SLLAB shares one Postgres database, hosted on Supabase. The iOS apps read and write it directly, through access rules enforced by the database. Operations that need a secret, such as sending SMS, run on a server. Figure 1 shows the system as it is today, with planned components dashed.

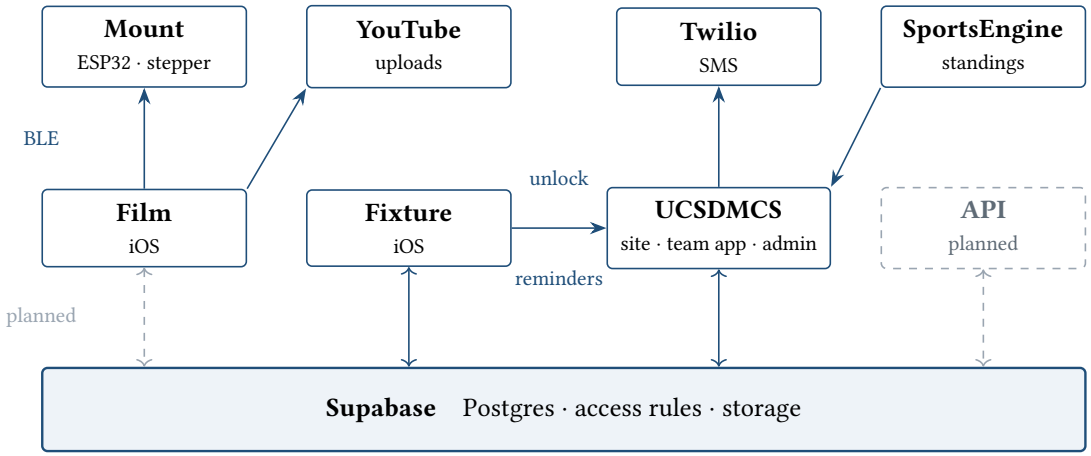


Figure 1: The system today. Fixture and the website share the database; Fixture calls the website for the two operations that need a server. Film does not yet use the shared backend.

Component	Stack	Role
Fixture by SLLAB	SwiftUI, supabase-swift	Player and captain app
UCSDMCS by SLLAB	Next.js 16, React 19, Tailwind 4	Public site, web team app, admin, server endpoints
Film by SLLAB	SwiftUI, AVFoundation, Vision, Core ML, Core Bluetooth	Recording and tracking
Hardware by SLLAB	ESP32 (Arduino), TMC2209, NEMA 17, EasyEDA	Pan base and its controller
Detection model	Ultralytics YOLO26s, exported to Core ML	On-device detection
Database	Supabase (Postgres 17)	Single source of truth

Table 1: Components and their technology.

3 Fixture by SLLAB

Fixture is the app the team uses every day. Its design constraint was adoption: a club cannot make 30 students create accounts, so joining had to take seconds.

3.1 Players

A player joins with a team code from their captain, picks their name from the roster, confirms it, and adds a phone number. From then on, the app opens to the season's schedule: practices and games, each with its field, a directions link, and the player's own response. Marking going or not going is one tap, and each event shows who else is coming.

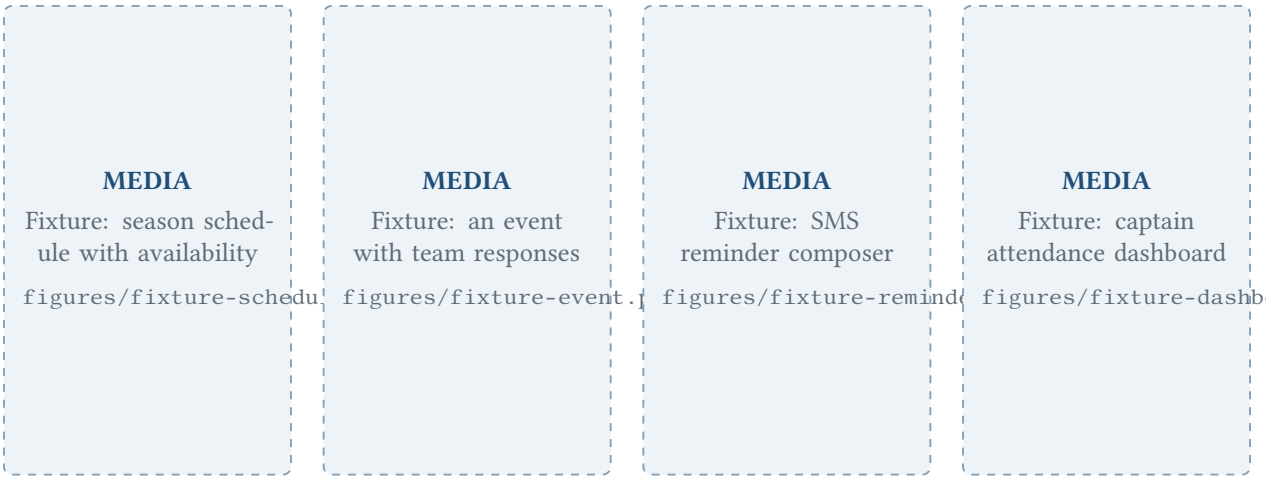


Figure 2: Fixture by SLLAB on iOS.

3.2 Captains

Captain mode unlocks with a separate code, which the server exchanges for a signed token. The app keeps the token in the Keychain and sends it with requests that the server performs on a captain's behalf, such as sending reminders. Captains can:

- create games and practices, including a repeating practice schedule: choose the weekdays and a date range, and every session is created at once;
- edit and delete events, and see each event's responses grouped into going, not going and no response;
- send an SMS reminder to selected players, for example everyone who hasn't responded;
- track attendance across the season.

Reminders never expose phone numbers. The app sends the server roster IDs and a message, not numbers. The server looks up the numbers, sends the texts through a Twilio Messaging Service, which also handles STOP and HELP replies, and returns how many were delivered. The numbers never reach the captain's device, and nobody outside the roster can be texted through the endpoint.

Attendance is ranked by what a captain needs to act on. For a player with n events in the selected period, of which they declined d and ignored u, the dashboard orders players by

$$\text{issue score} = 1.5\,\frac{u}{n} + \frac{d}{n}.$$

No response is weighted more heavily than declining: a no lets a captain plan, but silence doesn't. Each player's page lists the events they attended and the ones they missed, and why.

3.3 Shipping concerns

- **Universal links.** Reminder texts contain a link such as `app.ucsdmcs.com/events/<id>`, which opens the event directly in the app, or on the web if the app isn't installed.
- **Mandatory updates.** The database stores a minimum supported version. Older builds show an update screen. The check fails open, so if it can't reach the server, players are never locked out.

- **Tests.** Unit tests cover the models, JSON decoding against the database's shapes, availability grouping, attendance statistics and the data service.

4 UCSDMCS by SLLAB

The club's website serves three audiences from one Next.js deployment: the public, players using Fixture on the web, and officers maintaining the season's data.

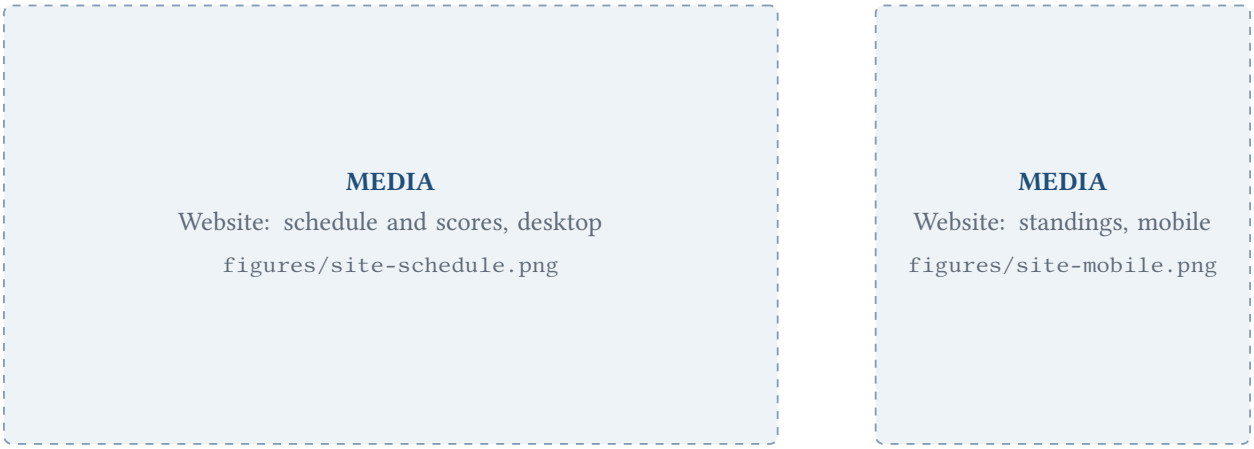


Figure 3: UCSDMCS by SLLAB at ucsdmcs.com.

4.1 The public site

The schedule shows each game with the opponent's crest, home or away, the field, and the result, under a season summary: record, win percentage, streak, home and away records, and goal difference. Standings come from the league's SportsEngine API. The roster, tryout information with a registration QR code, and the club's description are also on the site. The schedule is also published as an RFC 5545 calendar feed, so a game added in the admin dashboard appears in subscribers' calendars without anyone re-sharing it.

Caching follows how the data changes. Public pages are cached with no expiry and invalidated by a single tag whenever the admin dashboard writes, because club data is correct until someone changes it. Standings are the exception: league admins enter results on their own schedule, so they are fetched at most once a day.

4.2 The team app and the admin dashboard

The web team app is a port of Fixture, served at `app.ucsdmcs.com`. One deployment serves both hosts. A proxy step reads the request's host and serves the team section from the root, so the club site and the app have separate URLs without separate deployments.

The web and iOS clients share one session format: a payload signed with HMAC-SHA256. The website keeps it in an HTTP-only cookie; the app sends the same token as a bearer header. One format and one verification path serve both clients.

The admin dashboard lets officers edit teams and crests, the roster, the schedule, practices, socials, tournaments and fields. Writes happen in server actions, which check the session themselves rather than relying on the page layout, because a server action can be called directly.

5 Data model

The schema separates durable identity from seasonal participation, and generic events from their type-specific details. Figure 4 shows the core tables.

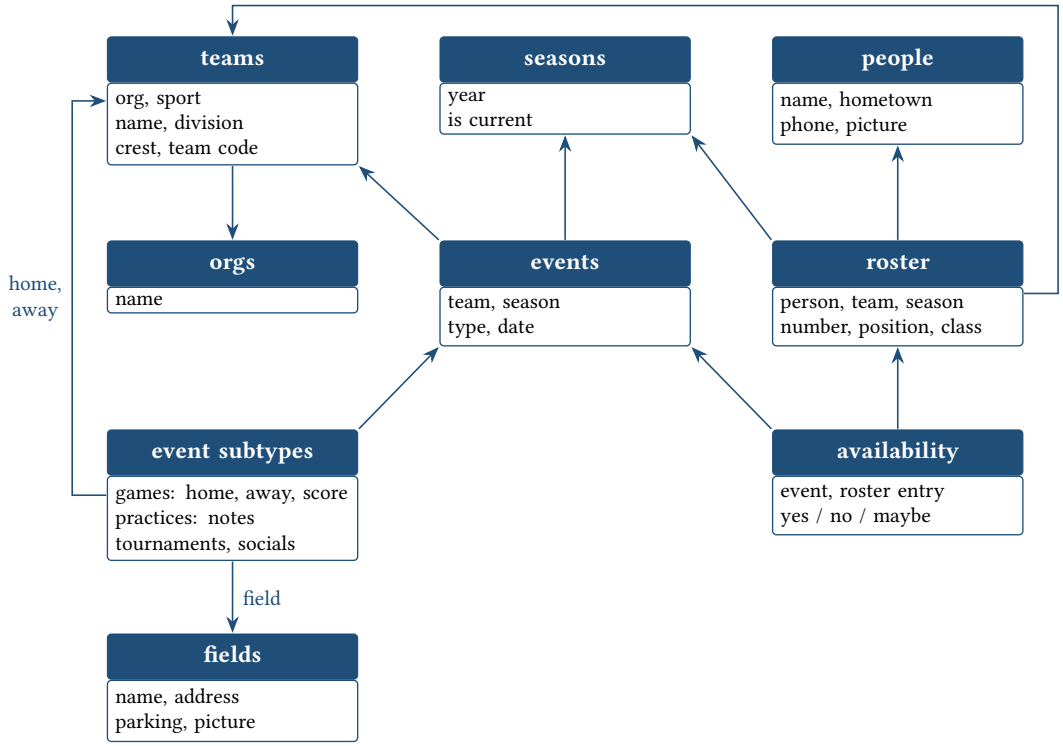


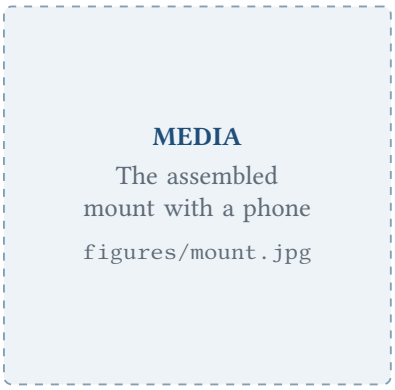
Figure 4: Core tables; arrows point from a row to the row it references. Games, practices, tournaments and socials each have their own table keyed by the event, grouped here.

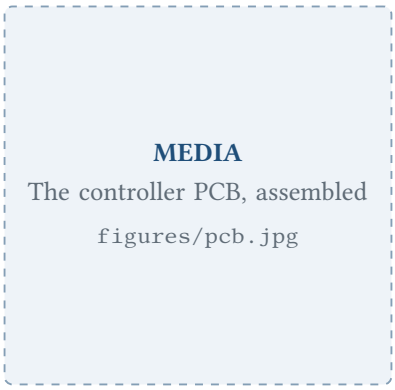
- **People are not roster entries.** A person exists once; a roster entry is that person on one team in one season, with that season's number, position and class. A player's history, and later their career statistics and alumni status, follow from this.
- **Events are a supertype.** Every event has a team, season, type and time, so the schedule, availability and reminders work the same for all of them. Details specific to a game or a practice live in subtype tables keyed by the event.
- **Availability belongs to a roster entry and an event**, with a uniqueness constraint on the pair, so a response is an upsert and can never be duplicated.
- **Games name two teams, not "us" and an opponent.** A game reads as home or away depending on whose side it is viewed from, so the same row serves both clubs once both are on the platform.
- **Constraints live in the database.** Allowed positions, classes and statuses are check constraints, and a partial unique index allows only one current season.

Some of these were right for one team and must change for many. The single current season is global, not per team; positions are soccer's; and joining relies on shared team and captain codes rather than personal accounts. Section 8 covers how each changes.

6 Film by SLLAB and the tracking mount

Film turns a phone into an unattended camera operator. The phone records on its ultra-wide camera while a detection model finds play in the frame, and a motorized base turns to keep it centered.





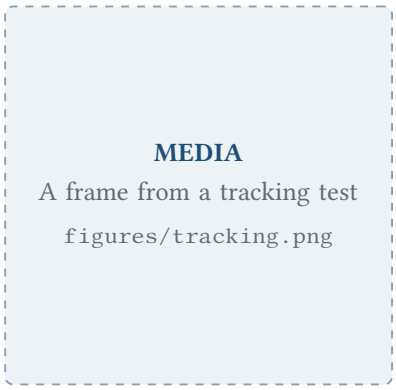


Figure 5: Hardware by SLLAB.

6.1 The control loop

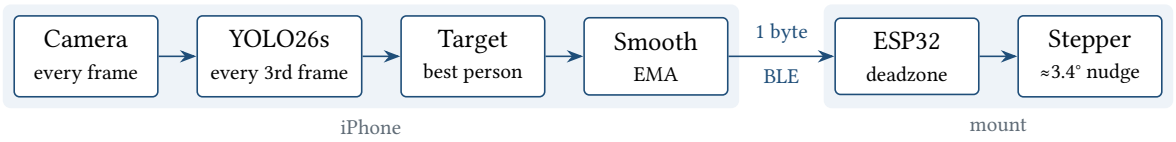


Figure 6: From camera frame to motor.

Detection. Frames go to a YOLO26s model through Vision and Core ML. Only every third frame is processed, keeping inference within budget while the phone also records. The target is the most confident person detection above 0.4 confidence.

Error signal. With $x_c \in [0, 1]$ the target's horizontal center in normalized image coordinates, the offset from the center of the frame is

$$o_t = 2\,(x_c - 0.5) \in [-1, 1],$$

smoothed with an exponential moving average so a single noisy detection doesn't swing the camera:

$$s_t = 0.7\, s_{t-1} + 0.3\, o_t.$$

When nothing is detected, the average resets, and the base holds still until play is found.

Protocol. The smoothed offset is quantized to one signed byte, $b = \mathrm{trunc}(127\, s_t)$, and written over Bluetooth Low Energy without acknowledgement. One byte with no round trip keeps latency low, and a lost command is replaced by the next one a fraction of a second later.

Firmware. The ESP32 keeps only the newest command: commands that arrive while the motor is moving overwrite each other instead of queueing, so the mount never moves toward where the target *was*. Offsets with $|b| \le 15$, about 12% of full scale, are ignored, so the camera holds steady when play is roughly centered instead of hunting back and forth. Outside the deadzone, the motor moves 15 microsteps toward the target. The driver runs at 1/8 microstepping on a 200-step motor, so each move is

$$\frac{15}{200 \times 8} \times 360^\circ \approx 3.4^\circ.$$

6.2 Hardware

The base is a NEMA 17 stepper driven by a TMC2209, whose quiet mode keeps motor noise off the recording's audio. The controller is a custom two-layer, through-hole PCB of about 45 × 65 mm, designed in EasyEDA, with the ESP32 and the driver in header sockets so either can be swapped without soldering. A 12 V supply feeds both the motor and the ESP32, with a 100 µF bulk capacitor to absorb current spikes when the motor steps. The housing, shaft coupler and phone holder are 3D-printed. The design went from a hand-wired breadboard to the printed base and then the PCB, and the enclosure and board were refined with a mechanical engineer and a systems engineer. The complete system costs about $400, against roughly $2,000 for commercial auto-tracking cameras.

6.3 Recording, library and upload

Film records on a timer or immediately, keeps videos and thumbnails in its own library rather than the camera roll, excludes them from iCloud backup, and stores metadata with SwiftData. Videos upload to YouTube as unlisted through the YouTube Data API's resumable protocol, and the link is saved with the video.

6.4 The detection model

The shipping model is YOLO26s, tracking people. To track the ball, the model is being fine-tuned with Ultralytics on a public Roboflow soccer-ball dataset, at 640 × 640 for 100 epochs on the Apple GPU. A second public dataset, labelling ball, goalkeeper, player and referee, is prepared for later work. The result is exported to Core ML with non-maximum suppression built in, so the app receives final detections.

6.5 Before release

Film is a working prototype. Before release, upload authorization moves to the platform's API, so each team uploads to its own channel and the app holds no credentials. Uploads must stream from disk instead of loading whole files into memory, since full-length matches are large. Tracking moves from the most confident player to the ball. And each video should be linked to its game through the shared backend, so film appears in Fixture.

7 Engineering practice

- **One repository.** The project began as five repositories. They were merged into one, preserving all of their history, so a schema change and every client it affects can land in one commit. Each app keeps its own folder and toolchain; shared code goes in packages when a second consumer needs it.
- **The schema is code.** The live database was captured as a baseline migration. From then on, every change is an ordered SQL file applied with the Supabase CLI, never an edit in the dashboard. The CLI version is pinned, because a major release changed the default target of database commands from the remote project to the local one.
- **Secrets stay out of git.** Every repository's history was scanned for keys before the merge. Only templates of required variables are committed, never values. The server's secret database key cannot reach a browser bundle: the module that holds it is marked server-only, which makes importing it from client code a build error rather than a convention.

- **Large artifacts stay out of git.** Training data, runs and weights are ignored; the repository holds the scripts and dataset definitions to re-create them.
- **CI on what changed.** A path-filtered workflow builds and lints each web app when it changes.

8 Roadmap

The next stage turns one team's tools into a platform that any club can join without a developer. Figure 7 shows the target structure.

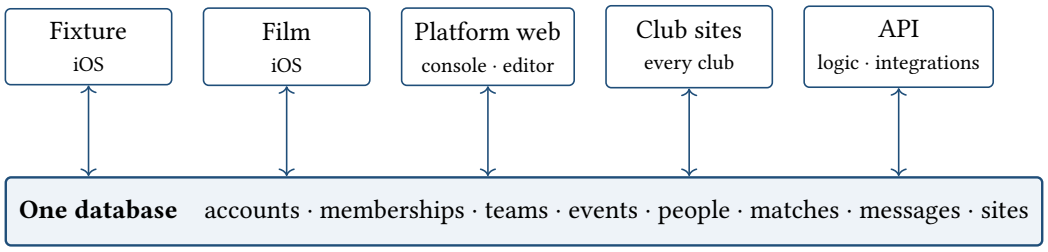


Figure 7: Target structure: every client on one backend, with platform logic in one API.

1. **Accounts and access.** Verified sign-in by phone or email, and a membership table relating each account to its teams and roles. Access rules in the database then scope every read and write to the caller's teams, and captain becomes a role granted by another captain instead of a shared code. Fixture's mandatory-update mechanism moves every player onto the new version before the old paths are removed.
2. **A platform API.** Server operations move from the club's website into a dedicated service with versioned routes, so no app depends on a club's domain. Messaging becomes a subsystem: templates, audience segments, scheduled sends, consent and delivery records.
3. **Multi-team data.** Seasons per team, sport-specific positions, and a shared directory of schools, teams, crests, fields and leagues that every club references instead of copying. Each new club then improves the data for the others.
4. **Matches and statistics.** Lineups and match events, from which player profiles, career statistics and standings follow.
5. **Club sites as data.** One deployment renders every club's site from its domain, with content stored as typed blocks and themed with the club's colors and crest. The UC San Diego site becomes the first set of rows instead of a codebase.
6. **Graphics, gear and dues.** Matchday and result graphics generated from match data; kit orders that collect sizes against roster numbers; dues collected through each club's own payment account.
7. **Film on the platform.** Videos attached to their events, from Film or any other source, and visible in Fixture.

9 Timeline

2026	Milestone
April	Film begins: camera capture and live preview.
June	Detection runs end to end, from frame capture to bounding boxes, on device.
July	The phone steers the mount over Bluetooth.
August	Recording moves to the ultra-wide camera and tracks at 10–15 ft. In-app library and YouTube upload. sllab.io launches.
September	The club website launches with the admin dashboard, live standings and tryouts. Fixture enters beta (1.0.0), and its web version follows.
October	SMS reminders and Keychain-backed captain sessions. Attendance tracking for captains. A new schema, unit tests and mandatory updates.
November	Past events in Fixture. The mount's firmware, PCB and CAD are published. The five repositories are merged into one.

Table 2: Development history, from the projects' commit logs.

AI Disclosure

Parts of this codebase and this write-up were written with the help of AI. All files were reviewed by a human, but they are provided as-is, with no guarantee that they are correct, secure, or fit for any particular purpose.